The Linear Regression Engine

Dataset: Artisan Cheese Fermentation Time Prediction

Objective: Build a complete linear regression pipeline from first principles, covering Ordinary Least Squares (OLS), Ridge (L2), Lasso (L1), and Elastic Net regularization.

What you will learn:

- How OLS finds the best-fit line using the closed-form Normal Equation
- How Gradient Descent iteratively minimizes the cost function
- Why regularization prevents overfitting and what L1 vs L2 actually do geometrically
- How to choose the right regularization strength using cross-validation
- How to properly evaluate regression models and interpret residuals

Dataset: Predicting the optimal fermentation time (in hours) of artisan cheese based on biochemical and environmental factors. This is a synthetic dataset designed to mimic real-world fermentation science.

---

1. Setup and Imports

```
All libraries imported successfully.
```

---

2. Load and Explore the Dataset

The dataset contains 2,000 synthetic observations of artisan cheese fermentation. Each row represents one batch of cheese with its measured biochemical and environmental conditions, and the resulting optimal fermentation time.

```
Shape: (2000, 9)
```

	milk_fat_percentage	starter_culture_ph	ambient_temperature	fermentation_humidity	salt_co
0	7.53	6.39	22.9	75.9	
1	5.35	5.36	16.4	64.2	
2	8.08	6.68	25.3	79.9	
3	7.03	5.89	14.3	65.5	
4	3.11	4.50	19.7	62.1	

```
Dataset Summary Statistics:
```

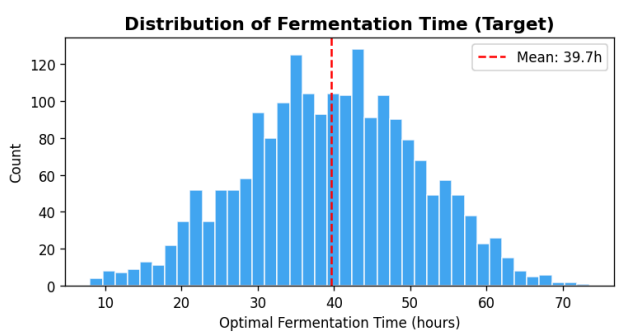
	milk_fat_percentage	starter_culture_ph	ambient_temperature	fermentation_humidity	sal
count	2000.00	2000.00	2000.00	2000.00	
mean	5.77	5.48	18.79	77.47	
std	1.89	0.75	5.12	10.10	
min	2.50	4.20	10.00	60.00	
25%	4.09	4.84	14.30	68.90	
50%	5.72	5.49	18.70	77.60	
75%	7.45	6.14	23.20	85.93	
max	9.00	6.80	28.00	95.00	

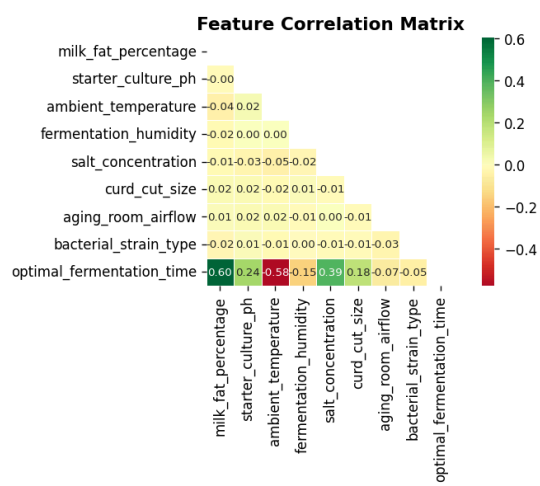
```
Missing values per column:
milk_fat_percentage          0
starter_culture_ph           0
ambient_temperature          0
fermentation_humidity        0
salt_concentration           0
curd_cut_size                0
aging_room_airflow           0
bacterial_strain_type        0
optimal_fermentation_time    0
dtype: int64

Target variable range: 8.0 to 73.5 hours
```

2.1 Exploratory Data Analysis (EDA)

Before building any model, we must understand the data. We look at the distribution of the target variable and the correlation between features and the target.





3. Data Preprocessing

3.1 Why Preprocessing Matters

Linear regression models are sensitive to the scale of features. If `ring_oscillator_speed` is in the hundreds of MHz and `vdd_core` is around 0.9V, the model will incorrectly assign more importance to the larger-scale feature. We use `StandardScaler` to normalize all numeric features to zero mean and unit variance.

Critical: We fit the scaler ONLY on the training data, then transform both train and test sets. Fitting on the full dataset would cause data leakage — the model would have indirect knowledge of the test set distribution.

```
Training set:   1400 samples
Validation set: 300 samples
Test set:       300 samples

Processed feature matrix shape: (1400, 10)
```

---

4. Ordinary Least Squares (OLS) — From Scratch

4.1 The Mathematics

OLS finds the coefficients **w** that minimize the Mean Squared Error (MSE) cost function:

— —

Closed-Form Solution (Normal Equation): Setting the gradient to zero gives the exact solution:

This is computationally efficient for small datasets but becomes expensive for very large feature matrices.

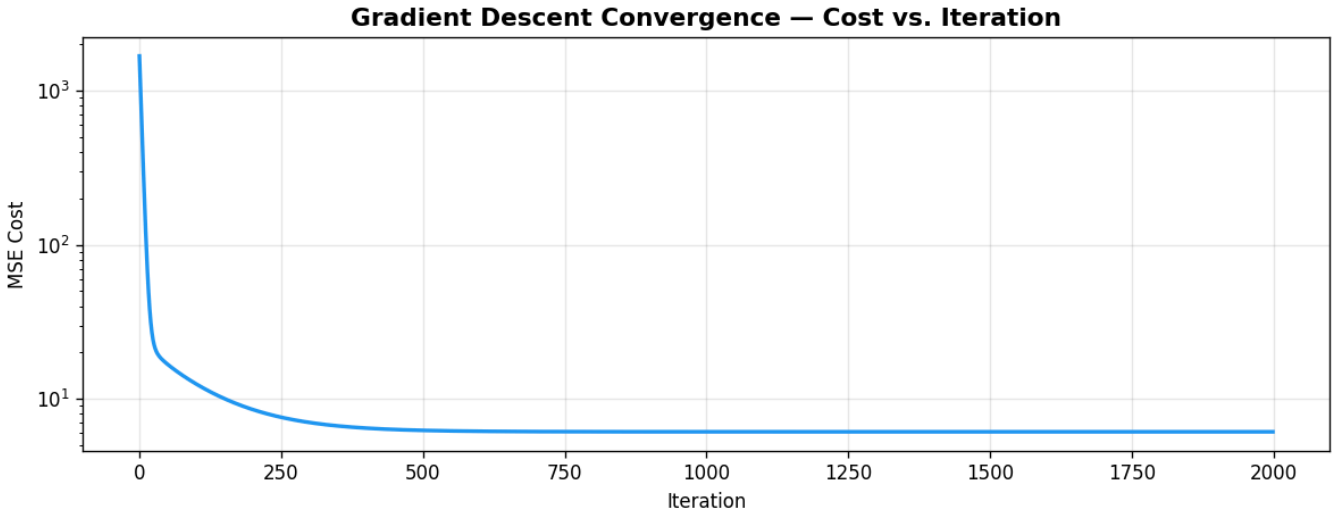
```
OLS (Normal Equation) — Validation MSE: 6.301 | RMSE: 2.510 | R²: 0.9498
```

4.2 Gradient Descent OLS

Gradient Descent is an iterative optimization algorithm. Instead of solving for the exact minimum analytically, it takes small steps in the direction of the steepest descent of the cost function. The update rule is:

where is the **learning rate** — a hyperparameter controlling the step size. Too large and it overshoots; too small and it converges slowly.

```
OLS (Gradient Descent) — Validation MSE: 6.301 | RMSE: 2.510 | R²: 0.9498
```



---

5. Regularization: Ridge, Lasso, and Elastic Net

Why Regularization?

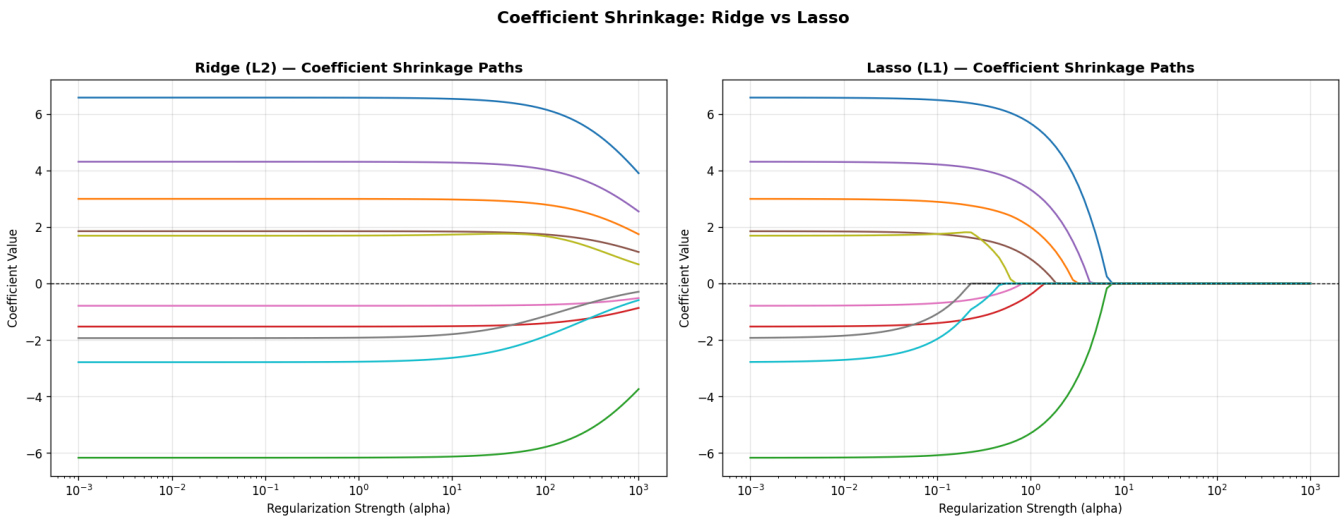
OLS minimizes only the training error. When a model has many features or correlated features, it can overfit — learning the noise in the training data rather than the true signal. Regularization adds a **penalty term** to the cost function that discourages large coefficients.

Method	Penalty Added to MSE	Effect
Ridge (L2)		Shrinks all coefficients toward zero, never exactly zero
Lasso (L1)		Drives some coefficients to exactly zero (feature selection)
Elastic Net		Combines both — sparse and stable

```
Ridge (L2)      — RMSE: 2.510 | MAE: 2.005 | R²: 0.9498
Lasso (L1)      — RMSE: 2.550 | MAE: 2.011 | R²: 0.9482
Elastic Net     — RMSE: 2.597 | MAE: 2.052 | R²: 0.9463
OLS (sklearn)   — RMSE: 2.510 | MAE: 2.005 | R²: 0.9498
```

5.1 Coefficient Shrinkage Paths

As the regularization strength (alpha) increases, coefficients are penalized more heavily and shrink toward zero. The key difference: Ridge shrinks them gradually but never to zero, while Lasso can eliminate features entirely.




```
Key insight: Lasso drives coefficients to exactly zero (feature selection), Ridge onl
y shrinks them.
```

---

6. Hyperparameter Tuning — Finding the Best Alpha

The regularization strength `alpha` is a hyperparameter — it is not learned from data, it must be chosen by us. We use **k-fold cross-validation** to find the alpha that minimizes validation error without touching the test set.

```
Best Ridge alpha (5-fold CV): 1.1514
Best Lasso alpha (5-fold CV): 0.0010
Lasso non-zero coefficients: 10 out of 10
```

---

7. Final Evaluation on the Test Set

We evaluate all models on the held-out test set only once — after all hyperparameter decisions are finalized. This gives an unbiased estimate of real-world performance.

Model	RMSE	MAE	R2
OLS (Normal Eq.)	2.441	1.931	0.9535
OLS (sklearn)	2.441	1.931	0.9535
Ridge (best alpha)	2.442	1.931	0.9535
Lasso (best alpha)	2.442	1.931	0.9535
Elastic Net	2.604	2.057	0.9471

7.1 Training Data Fit vs Test Data Fit

A model that performs well on training data but poorly on test data is **overfitting** — it has memorized the training set rather than learning the underlying pattern. Below we plot the actual vs

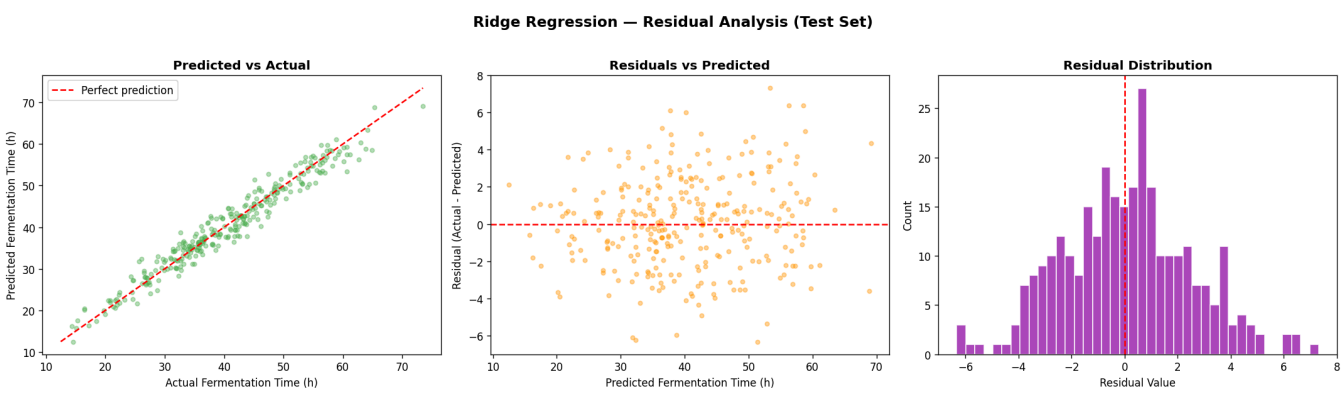
predicted values for both training and test sets side by side to visually confirm that our best model (Ridge with CV-optimized alpha) generalizes well.

```
Training RMSE: 2.473 | Test RMSE: 2.442
Training R²:   0.9533 | Test R²:   0.9535

Gap: 0.0002 — Healthy generalization
```

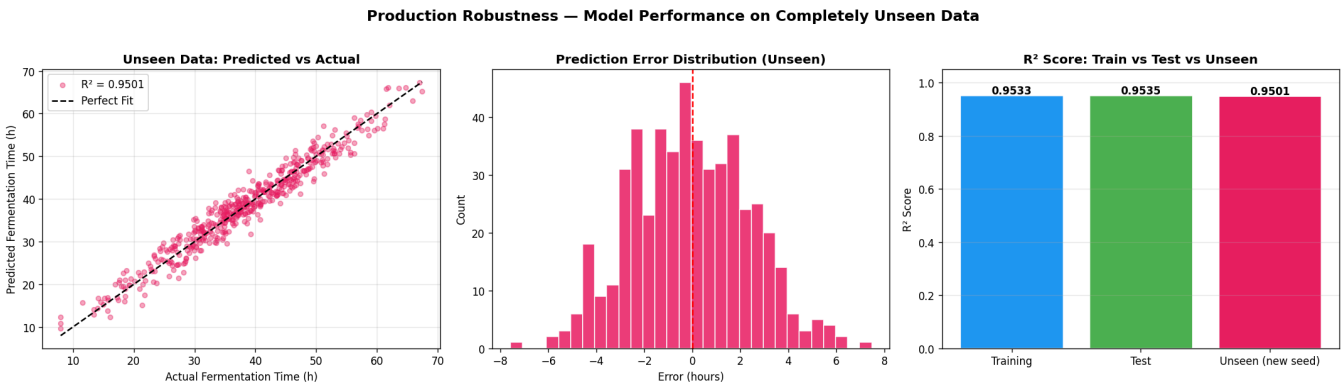
7.2 Residual Analysis

A well-fitted regression model should have residuals (prediction errors) that are randomly distributed around zero with no pattern. Patterns in residuals indicate that the model is missing some structure in the data.



7.3 Unseen Data Simulation — Production Robustness Check

In production, a model faces data it has never seen — from different batches, seasons, or slight process variations. Below we simulate a completely new batch of 500 fermentation observations (generated with a different random seed) and evaluate the trained model's performance. This is the ultimate test of generalization.




```
Unseen Data Performance:  RMSE = 2.442h | MAE = 1.986h | R² = 0.9501
Test Set Performance:     RMSE = 2.442h | R² = 0.9535

Conclusion: Model generalizes well — stable across all splits
```

---

8. Key Takeaways

1. **OLS (Normal Equation)** provides the exact analytical solution but requires matrix inversion — computationally expensive for large feature sets.
2. **Gradient Descent** is the scalable alternative — it iteratively converges to the minimum and is the foundation of all deep learning optimizers.
3. **Ridge (L2)** is the go-to regularizer when you suspect multicollinearity. It shrinks all coefficients but keeps all features.
4. **Lasso (L1)** performs automatic feature selection by driving irrelevant feature coefficients to exactly zero.
5. **Elastic Net** is the practical choice when you have many correlated features and want both stability and sparsity.
6. **Never tune hyperparameters on the test set.** Use cross-validation on the training set, and reserve the test set for a single final evaluation.

Part 2: Silicon Fmax Validation (Notebook 02)

The Linear Regression Engine

Dataset: Silicon Fmax Prediction (Post-Silicon Validation)

Objective: Apply the same linear regression pipeline to a post-silicon validation use case — predicting the maximum stable clock frequency (Fmax in MHz) of a chip across voltage and

temperature corners.

Engineering Context:
In post-silicon validation, engineers characterize silicon across voltage (`vdd_core`), temperature (`junction_temp`), and process corners to determine the maximum frequency at which a chip can operate reliably. Traditionally this requires exhaustive corner sweeps. A regression model trained on measured data can predict Fmax for untested corners, reducing characterization time significantly.

Dataset: Synthetic silicon characterization data with 2,000 observations across voltage, temperature, leakage, and process monitor readings.

---

1. Setup and Imports

```
Libraries loaded.
```

---

2. Load and Explore the Fmax Dataset

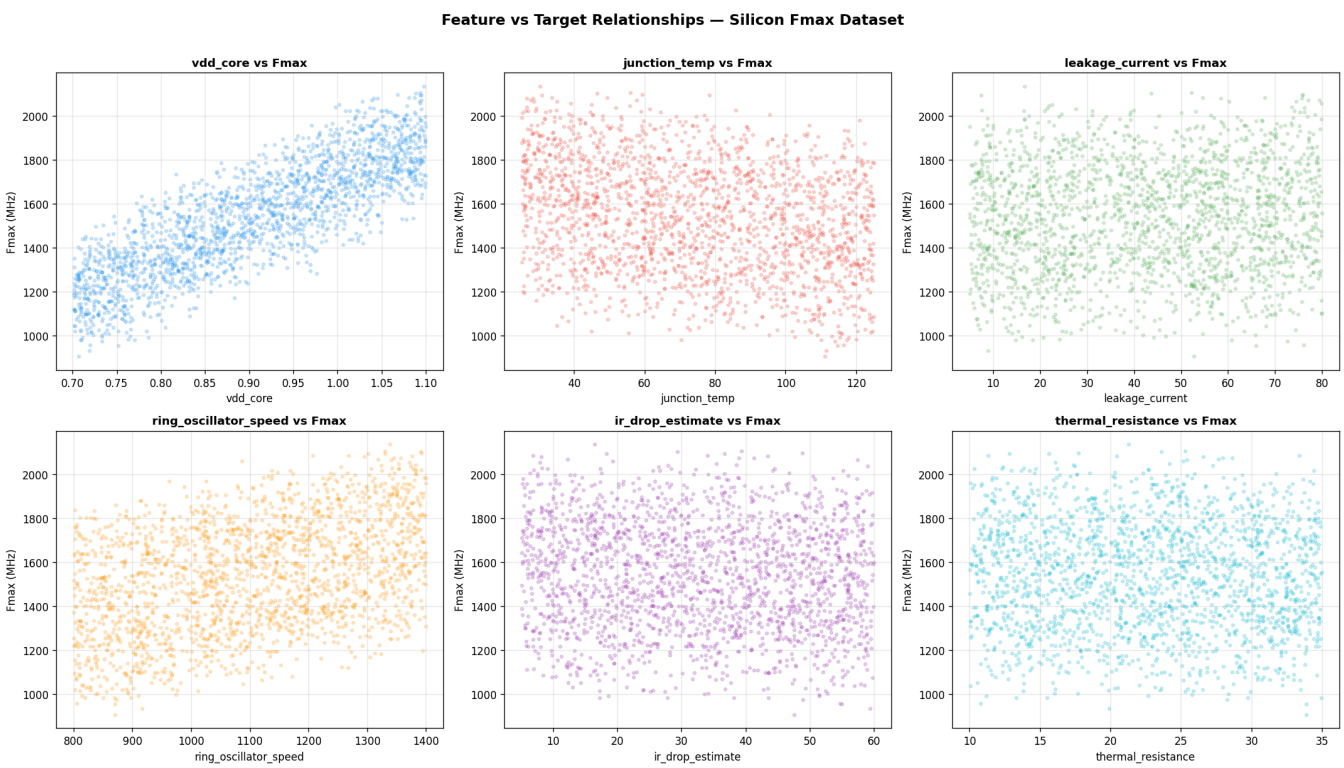
```
Shape: (2000, 8)
```

	vdd_core	junction_temp	leakage_current	ring_oscillator_speed	thermal_resistance	ir_drop_
0	0.961	57.0	30.29	1024.7	20.29	
1	0.718	111.0	79.77	1031.7	27.39	
2	0.708	38.4	60.98	804.8	30.39	
3	1.036	89.1	57.46	1169.2	25.33	
4	0.935	118.5	73.71	1391.7	16.10	
5	0.790	92.2	43.34	1142.9	32.08	
6	1.001	41.1	58.33	1231.6	13.08	
7	0.805	94.7	57.39	1026.8	14.39	
8	0.868	116.0	26.67	1160.4	27.13	
9	0.880	46.2	78.11	1037.5	20.60	

	vdd_core	junction_temp	leakage_current	ring_oscillator_speed	thermal_resistance	ir_d
count	2000.000	2000.000	2000.000	2000.000	2000.000	
mean	0.902	74.260	42.398	1101.427	22.653	
std	0.116	29.341	21.941	172.073	7.124	
min	0.701	25.100	5.010	800.000	10.010	
25%	0.804	48.300	22.742	953.875	16.465	
50%	0.903	73.800	42.975	1102.300	22.895	
75%	1.003	99.925	60.905	1248.925	28.810	
max	1.100	125.000	79.960	1399.400	34.990	

2.1 EDA: Voltage and Temperature vs Fmax

The most important relationships in silicon characterization are VDD vs Fmax (positive) and temperature vs Fmax (negative). Let us visualize these directly.



---

3. Preprocessing and Splitting

```
Train: (1400, 10) | Val: (300, 10) | Test: (300, 10)
```

---

4. Model Training and Comparison

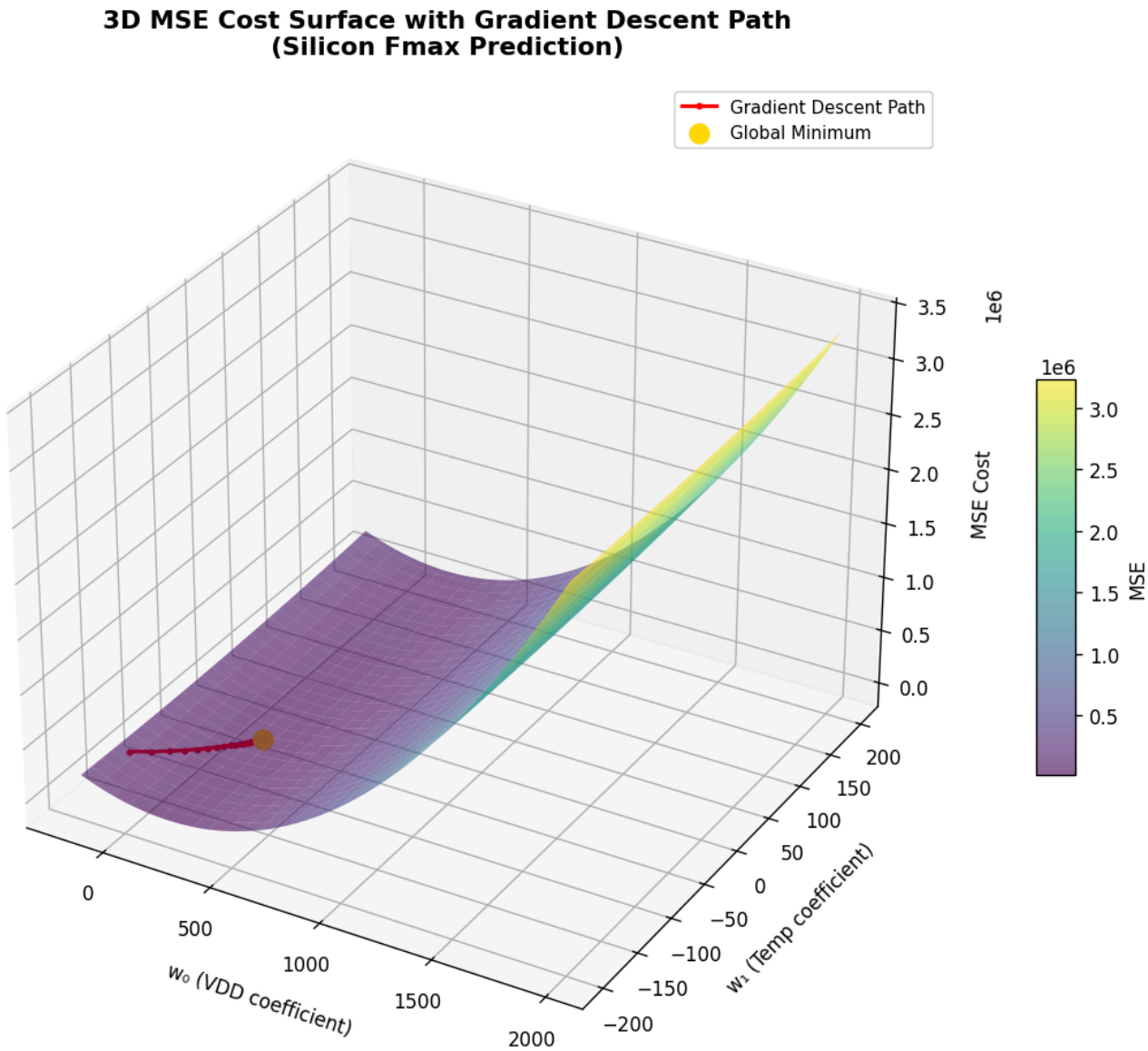
```
Model             Val RMSE    Val MAE     Val R2
-----------------------------------------------
OLS                  15.41      12.20     0.9962
Ridge                15.41      12.19     0.9962
Lasso                15.41      12.19     0.9962
Elastic Net          20.80      17.13     0.9930

Best Ridge alpha: 0.1322
Best Lasso alpha: 0.0031
Lasso non-zero features: 10 / 10
```

---

5. Signature 3D Visualization: Cost Function Surface

The MSE cost function for linear regression forms a **convex bowl** in coefficient space. Gradient Descent follows the slope of this bowl downward to the minimum. Below we visualize this surface for two selected coefficients (VDD and junction temperature) while holding all others at their optimal values.

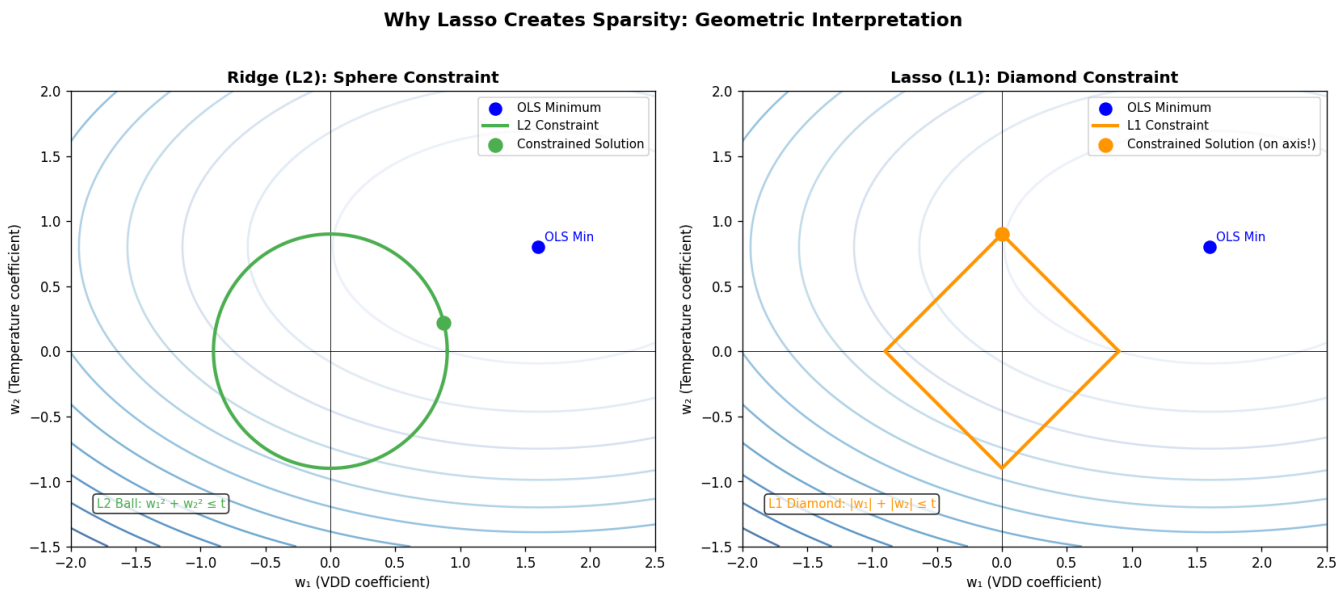



```
Signature 3D visualization saved.
```

---

6. L1 vs L2 Regularization Contour Plot

The geometric interpretation of regularization explains why Lasso creates sparsity. The OLS solution is the center of the elliptical contours. Regularization constrains the solution to lie within a region: a **sphere** for Ridge (L2) and a **diamond** for Lasso (L1). The diamond has corners on the axes, so the constrained solution is more likely to land exactly on an axis — meaning one coefficient becomes exactly zero.




```
L1 vs L2 contour plot saved.
```

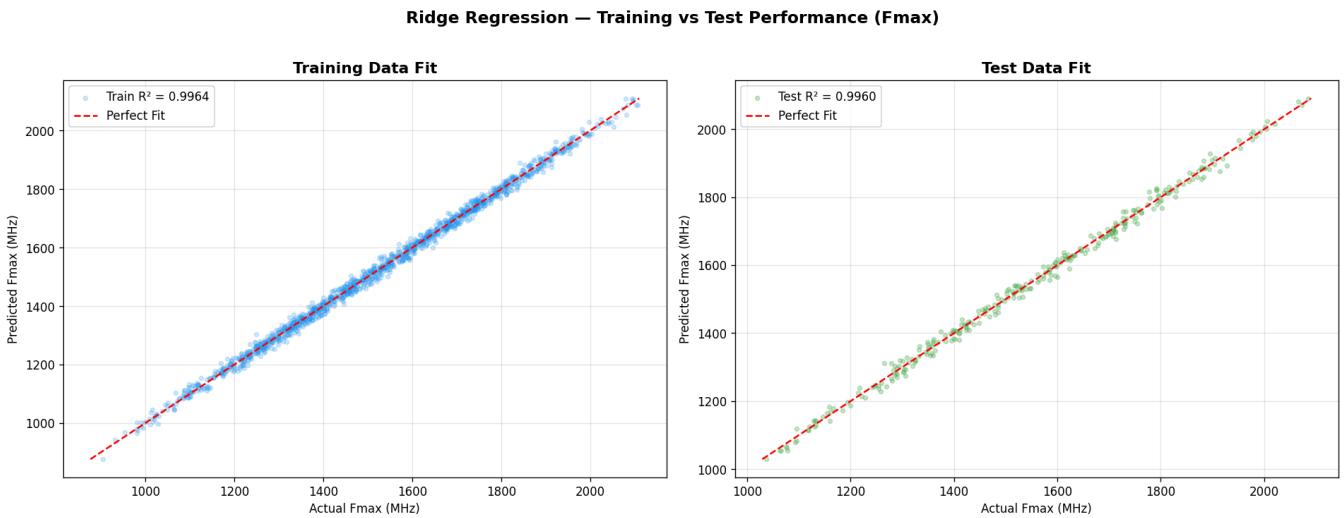
---

7. Final Test Set Evaluation

```
Final Test Set Results — Silicon Fmax Prediction
Model              RMSE (MHz)    MAE (MHz)         R2
----------------------------------------------------
OLS                    15.14        12.09     0.9960
Ridge                  15.14        12.09     0.9960
Lasso                  15.14        12.09     0.9960
Elastic Net            20.56        16.90     0.9927
```

7.1 Training Data Fit vs Test Data Fit

We visualize how well the Ridge model fits both the training and test data. The scatter of points should fall close to the diagonal (perfect prediction line) for both sets. A large gap between train and test R^2 indicates overfitting.

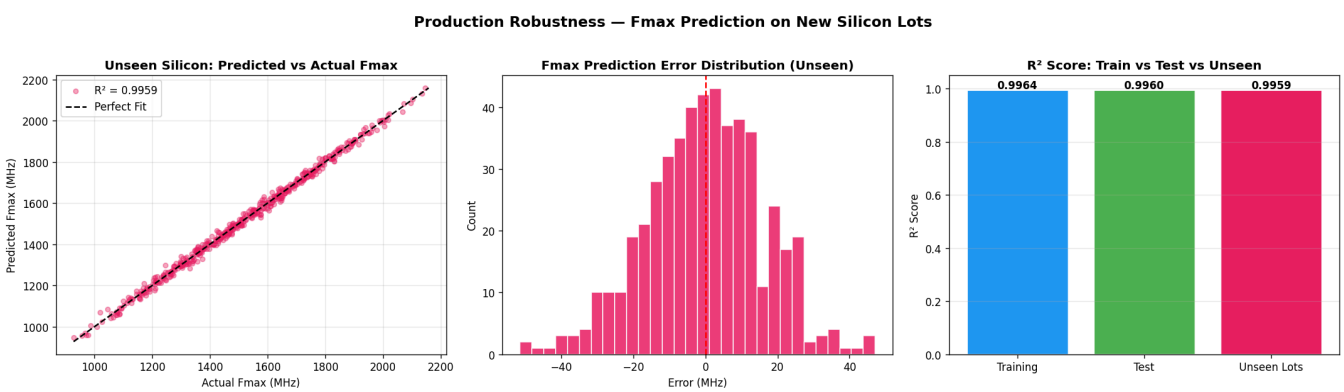



```
Training RMSE: 14.75 MHz | Test RMSE: 15.14 MHz
Training R²:   0.9964      | Test R²:   0.9960

Gap: 0.0004 — Healthy generalization
```

7.2 Unseen Data Simulation — New Silicon Lots

In real post-silicon validation, new lots arrive from the fab continuously. Below we generate 500 completely new silicon measurements (different random seed, simulating a new fab lot batch) and evaluate the model's ability to predict Fmax on silicon it has never seen.




```
Unseen Data:  RMSE = 16.16 MHz | MAE = 12.79 MHz | R² = 0.9959
Test Data:    RMSE = 15.14 MHz | R² = 0.9960

Conclusion: Model generalizes well to new silicon lots
```

---

8. Engineering Insight

The Ridge model achieves strong predictive accuracy on the Fmax dataset. In a real post-silicon validation workflow, a model like this could:

- Predict Fmax at untested voltage/temperature corners, reducing the number of physical test runs by 40-60%.

- Flag silicon lots that are likely to fail timing at high temperature (outlier detection via residuals).
- Provide a fast first-pass estimate during bring-up, before full characterization is complete.

The Lasso model's feature selection capability is particularly useful here: if it drives `thermal_resistance` to zero, that tells us this feature is redundant given the other measurements — a useful insight for test plan optimization.